

SQL

Shahram Ghandeharizadeh
CSCI 585

SQL Data Definition Language

A "What"-oriented language. The system is responsible for "How".

It allows the specification of not only a set of relations but also information about each relation, including:

- The schema for each relation.
- The domain of values associated with each attribute.
- Integrity constraints
- The set of indices to be maintained for each relations.
- Security and authorization information for each relation.
- The physical storage structure of each relation on disk.

Domain Types in SQL

- **char(*n*)**. Fixed length character string, with user-specified length *n*.
- **varchar(*n*)**. Variable length character strings, with user-specified maximum length *n*.
- **int**. Integer (a finite subset of the integers that is machine-dependent).
- **smallint**. Small integer (a machine-dependent subset of the integer domain type).
- **numeric(*p,d*)**. Fixed point number, with user-specified precision of *p* digits, with *n* digits to the right of decimal point.
- **real, double precision**. Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(*n*)**. Floating point number, with user-specified precision of at least *n* digits.

SQL INTRODUCTION

- For the purposes of the following, assume the existence of these two relations:
Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)
- **Structured Query Language (SQL)** consists of four basic commands: Select, Insert, Update, and Delete.
- SQL provides commands to change the state of database: table creation/deletion, **insert**, **delete**, and **update**

CREATE TABLE COMMAND

- **Emp** (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)
- The allowed integrity constraints include:
 - Primary key (Aj1, Aj2, ..., Ajm): states that attributes Aj1, Aj2, ..., Ajm constitute the primary key for relation r: non-null and unique.
 - Check(P): states that every record in the relation must satisfy the specified predicate P, e.g., P = “age > 10”
- Create table Emp (SS# int, name char(15), age int, salary int, dno int, **primary key** (SS#), **check** (salary > 24000))
- Create table Dept (dno int, dname char(15), floor int, mgrss# int, **primary key** dno)

Basic Query Structure

- SQL is based on set and relational operations with certain modifications and enhancements
- A typical SQL query has the form:

select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

- A_i represents an attribute
- R_i represents a relation
- P is a predicate.

- The result of an SQL query is a relation.

SELECT COMMAND

- The select command has the following syntax:
select [**distinct**] target-list
from tuple variable list
where qualification
[**order by** target list subset]
[**having** set-qualification]
the items in square brackets ([]) are electives.
- Qualification list can be a boolean combination (and, or, not) of either selection and join clauses.
- A *selection clause* is a comparison between an indexed tuple variable and a constant. A comparison operator might be $\{=, \neq, \leq, \geq, >, <\}$.

SELECT COMMAND (Cont...)

- A *join clause* is a comparison between two indexed tuple variable. Once again, a comparison operator might be $\{=, \neq, \leq, \geq, >, <\}$. In the following example, `e.dno=d.dno` is a join clause while `d.dname='Toy'` is a selection clause:

select e.name

from Emp e, Dept d

where e.dno = d.dno and d.dname = 'Toy'

select e.name
from Emp e, Dept d
where e.dno = d.dno and d.dname = 'Toy'

Result = ?

Emp

ssnum	name	age	salary	dno
123	Mary	22	55000	3
323	Joe	33	34000	1
324	John	40	55000	2
333	Mike	25	60000	1

Dept

dno	dname	floor	mgrssnum
1	shoe	1	323
2	toy	1	323
3	men	2	323
4	women	2	324

ssnum	name	age	salary	dno
123	Mary	22	55000	3
323	Joe	33	34000	1
324	John	40	55000	2
333	Mike	25	60000	1

dname = “Toy”

dno	dname	floor	mgrssnum
2	toy	1	323

ssnum	name	age	salary	dno
123	Mary	22	55000	3
323	Joe	33	34000	1
324	John	40	55000	2
333	Mike	25	60000	1

dname = “Toy”

dno	dname	floor	mgrssnum
2	toy	1	323

Emp.dno = Dept.dno and d.dname = “Toy”

ssnum	name	age	salary	dno	dname	floor	mgrss#
324	John	40	55000	2	Toy	1	323

SELECT COMMAND (Cont...)

- To eliminate duplicates from the result, use the keyword **distinct** in the target list. For example, the following query shows all distinct salaries above 50,000 with no duplicates:

```
select distinct e.salary  
from Emp e  
where e.salary > 50000
```

SELECT COMMAND (Cont...)

Conceptualize the execution of a select query as one of:

1. evaluate qualification to locate qualifying tuples,
2. project out columns of these tuples using the target-list,
3. make the resulting tuples into a relation and eliminate duplicates if necessary.

select [**distinct**] target-list
from tuple variable list
where qualification

Quick Review

Emp (SS#, name, age, salary, dno)

Dept (dno, dname, floor, mgrSS#)

- Name of those employees younger than 18 years old.

Quick Review

Emp (SS#, name, age, salary, dno)

Dept (dno, dname, floor, mgrSS#)

- Name of those employees younger than 18 years old.

```
SELECT  e.name
FROM    Emp e
WHERE   e.age < 18
```

Quick Review

Emp (SS#, name, age, salary, dno)

Dept (dno, dname, floor, mgrSS#)

- Name of those employees working on the first floor.

Quick Review

Emp (SS#, name, age, salary, dno)

Dept (dno, dname, floor, mgrSS#)

- Name of those employees working on the first floor.

```
SELECT  e.name
FROM    Emp e, Dept d
WHERE   e.dno = d.dno and d.floor=1
```

Quick Review

Emp (SS#, name, age, salary, dno)

Dept (dno, dname, floor, mgrSS#)

- Name of the employee managing the Toy department.

Quick Review

Emp (SS#, name, age, salary, dno)

Dept (dno, dname, floor, mgrSS#)

- Name of the employee managing the Toy department.

```
SELECT  e.name
FROM    Emp e, Dept d
WHERE   d.dname = Toy and d.mgrSS=e.SS#
```

Quick Review

Emp (SS#, name, age, salary, dno)

Dept (dno, dname, floor, mgrSS#)

- Name of the employee who are older than their manager.

Quick Review

Emp (SS#, name, age, salary, dno)

Dept (dno, dname, floor, mgrSS#)

- Name of the employee who are older than their manager.

```
SELECT  e.name
FROM    Emp emp, Dept d, Emp mgr
WHERE   e.dno=d.dno and d.mgrSS#=mgr.SS# and e.age>mgr.age
```

Relational Algebra + Query Tree

Shahram Ghandeharizadeh
CSCI 585

RELATIONAL ALGEBRA INTRODUCTION

- Assume the following two relations:
Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)
- Relational algebra is a procedural query language, i.e., user must define both “how” and “what” to retrieve. It is used to implement SQL.
- Relational algebra consists of a set of operators that consume either one or two relations as input. An operator produces one relation as its output.
- Unary operators include: select, project, and rename
- Binary operators include: cartesian product, equality join, natural join, join, semi-join, division, union, and set difference.

SELECT OPERATOR

- **Select (σ):** selects tuples that satisfy a predicate; e.g., retrieve the employees whose salary is 30,000

$\sigma_{\text{Salary}=30,000}(\text{Employee})$

- Conjunctive ($\wedge$) and disjunctive ($\vee$) selection predicates are allowed; e.g., retrieve employees whose salary is higher than 30,000 and are younger than 25 years old:

$\sigma_{\text{Salary}>30,000 \wedge \text{age}<25}(\text{Employee})$

- Note that only selection predicates are allowed. A selection predicate is either (1) a comparison ($=, \neq, \leq, \geq, <, >$) between an attribute and a constant (e.g., salary = 30,000) or (2) a comparison between two different attributes of the same relation (e.g., salary = age $\times$ 100).
- Note: This operator is different than the SELECT command of SQL.

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\sigma_{\text{Salary}=30,000}(\text{Employee})$

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\sigma_{\text{Salary}=30,000}(\text{Employee})$

SS#	Name	Age	Salary	dno
4	Kathy	30	30000	5

SQL?

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\sigma_{\text{Age} > 22}(\text{Employee})$

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\sigma_{\text{Age} > 22}(\text{Employee})$

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
4	Kathy	30	30000	5

SQL?

PROJECT OPERATOR

- **Project** (Π) retrieves a column. It is a unary operator that eliminate duplicates.

e.g., name of employees:

$\Pi_{\text{name}}(\text{Employee})$

e.g., name of employees earning more than 30,000:

$\Pi_{\text{name}}(\sigma_{\text{Salary} > 30,000}(\text{Employee}))$

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\Pi_{\text{age}}(\text{Emp})$

Age
24
20
22
30
4

SQL?

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\Pi_{\text{name,age}}(\sigma_{\text{Salary}=4000}(\text{Emp}))$

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\Pi_{\text{name,age}}(\sigma_{\text{Salary}=4000}(\text{Emp}))$

SS#	Name	Age	Salary	dno
5	Genius	4	4000	1

EXAMPLE

- Emp table:

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	3
3	Bob	22	27000	4
4	Kathy	30	30000	5
5	Genius	4	4000	1

- $\Pi_{\text{name,age}}(\sigma_{\text{Salary}=4000}(\text{Emp}))$

Name	Age
Genius	4

SQL?

RENAME OPERATOR

- Rename operator changes the name of its input table to its subscript,
 - $\rho_{e2}(\text{Emp})$
 - Changes the name of Emp table to e2

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN

- Equality join connects tuples from two relations that match on certain attributes. The specified joining columns are kept in the resulting relation.
 - $\Pi_{\text{name}}(\sigma_{\text{dname}='toy'}(\text{Emp} \bowtie_{(\text{dno})} \text{Dept}))$
- Natural join connects tuples from two relations that match on the specified common attributes
 - $\Pi_{\text{name}}(\sigma_{\text{dname}='toy'}(\text{Emp} \bowtie_{(\text{dno})} \text{Dept}))$
- How is an equality join between Emp and Dept using dno different than a natural join between Emp and Dept using dno?
 - Equality join: SS#, name, age, salary, Emp.dno, Dept.dno, ...
 - Natural join: SS#, name, age, salary, dno, dname, ...
- Join is similar to equality join using different comparison operators
 - $A \bowtie S \quad \text{op} = \{=, \neq, \leq, \geq, <, >\}$
att op att

EXAMPLE JOIN

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	1
3	Bob	22	27000	1
4	Kathy	30	30000	2
5	Genius	4	4000	1

EMP

dno	dname	floor	mgrss#
1	Toy	1	5
2	Shoe	2	1

Dept

- Equality Join, (Emp $\bowtie_{(dno)}$ Dept)))

SS#	Name	Age	Salary	EMP.dno	Dept.dno	dname	floor	mgrss#
1	Joe	24	20000	2	2	Shoe	2	1
2	Mary	20	25000	1	1	Toy	1	5
3	Bob	22	27000	1	1	Toy	1	5
4	Kathy	30	30000	2	2	Shoe	2	1
5	Genius	4	4000	1	1	Toy	1	5

EXAMPLE JOIN

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	1
3	Bob	22	27000	1
4	Kathy	30	30000	2
5	Genius	4	4000	1

EMP

dno	dname	floor	mgrss#
1	Toy	1	5
2	Shoe	2	1

Dept

- Natural Join, (Emp $\bowtie_{(dno)}$ Dept)))

SS#	Name	Age	Salary	dno	dname	floor	mgrss#
1	Joe	24	20000	2	Shoe	2	1
2	Mary	20	25000	1	Toy	1	5
3	Bob	22	27000	1	Toy	1	5
4	Kathy	30	30000	2	Shoe	2	1
5	Genius	4	4000	1	Toy	1	5

EXAMPLE JOIN

SS#	Name	Age	Salary	dno
1	Joe	24	20000	2
2	Mary	20	25000	1
3	Bob	22	27000	1
4	Kathy	30	30000	2
5	Genius	4	4000	1

EMP

Salary > 5 * salary

dno	dname	floor	mgrss#
1	Toy	1	5
2	Shoe	2	1

Dept

- Join, (Emp $\bowtie$ ρ_x (Emp))))
Salary > 5 * x.Salary

SS#	Name	Age	Salary	dno	x.SS#	x.Name	x.Age	x.Salary	x.dno
2	Mary	20	25000	1	2	Genius	4	4000	1
3	Bob	22	27000	1	3	Genius	4	4000	1
4	Kathy	30	30000	2	4	Genius	4	4000	1

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont...)

- Example, retrieve Joe's salary:

:

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont...)

- Example, retrieve Joe's salary:
 - $\Pi_{\text{Salary}}(\sigma_{\text{name}='Joe'}(\text{Emp}))$

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont...)

- Example, retrieve the name of Joe's manager:

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont...)

- Example, retrieve the name of employees who earn more than Joe:

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EQUALITY JOIN, NATURAL JOIN, JOIN, SEMI-JOIN (Cont...)

- Example: retrieve the name of employees who earn more than Joe:
 - $\Pi_{\text{name}}(\text{Emp} \bowtie_{(\text{sal} > \text{x.sal})} \sigma_{\text{name}='Joe'}(\rho_{\text{x}}(\text{Emp})))$

SQL?

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

EXAMPLE

- Assume a database with the following three relations:
Sailors (sid, sname, rating)
Boats (bid, bname, color)
Reserve (sid, bid, date)
- Query 1: Find the bid of red colored boats.

EXAMPLE

- Assume a database with the following three relations:
Sailors (sid, sname, rating)
Boats (bid, bname, color)
Reserve (sid, bid, date)
- Query 1: Find the name of sailors who have reserved a red boat.

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

- Employees working for the Toy department

Quick Review

Emp (SS#, name, age, salary, dno)
Dept (dno, dname, floor, mgrSS#)

- Employees working for the Toy department:

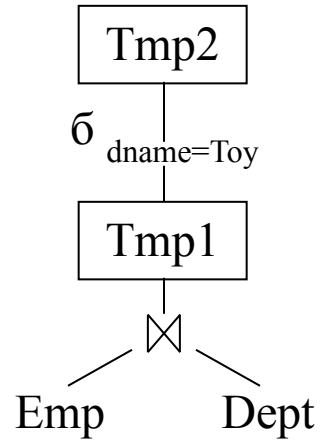
$$\text{TMP2} \leftarrow \sigma_{\text{dname}=\text{Toy}}(\text{Emp} \bowtie \text{Dept})$$

$$\text{TMP2} \leftarrow \text{Emp} \bowtie \sigma_{\text{dname}=\text{Toy}}(\text{Dept})$$

SQL?

Query Tree

- $TMP2 \leftarrow \sigma_{dname=Toy}(Emp \bowtie Dept)$



Query Tree

- $TMP2 \leftarrow Emp \bowtie \sigma_{dname=Toy}(Dept)$

